

k-NN y Naive Bayes

Dos algoritmos clásicos de clasificación que sientan las bases del aprendizaje automático moderno. En esta presentación exploraremos su funcionamiento, sus ventajas y sus limitaciones, con especial atención a su aplicación en tareas de minería de texto.

TEMA 7.3

APRENDIZAJE AUTOMÁTICO



Índice de contenidos

01

La idea de k-NN

Clasificación por proximidad y vecindad

03

Escalado de variables

Por qué la escala importa y cómo corregirla

05

Naive Bayes

Probabilidad, independencia y texto

02

Métricas de distancia

Euclidiana, Manhattan y otras

04

Elección de k

El hiperparámetro que lo cambia todo

06

Comparativa final

¿Cuándo usar cada algoritmo?

La idea de k-NN

k-Nearest Neighbors (k-NN) es uno de los algoritmos más intuitivos del aprendizaje automático. Su principio fundamental es sencillo: *dime con quién te juntas y te diré quién eres*. Para clasificar un nuevo ejemplo, el algoritmo busca los **k puntos más cercanos** en el conjunto de entrenamiento y asigna la clase mayoritaria entre ellos.

Sin entrenamiento explícito

k-NN es un algoritmo *lazy*: no construye un modelo durante el entrenamiento, sino que memoriza todos los ejemplos.

Decisión por votación

Los k vecinos votan y la clase con más votos gana. En caso de empate, se puede reducir k o usar pesos por distancia.

No paramétrico

No asume ninguna forma funcional para los datos, lo que lo hace flexible ante distribuciones complejas.

Métricas de distancia

La elección de la métrica de distancia es crítica en k-NN, ya que determina qué puntos se consideran «vecinos». Las más habituales son:

Distancia Euclidiana

La más común. Calcula la línea recta entre dos puntos en el espacio: $\sqrt{\sum (\mathbf{x}_i - \mathbf{y}_i)^2}$. Funciona bien cuando las escalas son similares y las variables son continuas.

Distancia Manhattan

Suma de las diferencias absolutas: $\sum |\mathbf{x}_i - \mathbf{y}_i|$. También llamada distancia «ciudad» o L1. Más robusta frente a *outliers* que la euclidiana.

Distancia Coseno

Mide el ángulo entre dos vectores, no su magnitud. Especialmente útil en texto, donde importa la **dirección** del vector de frecuencias de palabras, no su longitud.

Distancia de Minkowski

Generalización paramétrica que engloba la euclidiana ($p=2$) y la Manhattan ($p=1$). Permite ajustar la sensibilidad a diferencias grandes o pequeñas.

CONCEPTO CLAVE

La importancia del escalado

Si una variable tiene valores en el rango 0–1 y otra en el rango 0–10.000, la segunda **dominará completamente** el cálculo de distancias, haciendo que la primera sea irrelevante. El escalado previo es imprescindible para que todas las variables contribuyan de forma equilibrada.

Normalización (Min–Max)

Escala todos los valores al rango [0, 1]. Sensible a valores extremos.

Estandarización (Z-score)

Centra en 0 con desviación típica 1. Más robusta si hay *outliers*.

Ejemplo práctico

Clasificamos viviendas por precio y superficie. Sin escalar, el precio (en euros) eclipsa la superficie (en m²). Tras estandarizar, ambas variables contribuyen por igual al cálculo de vecindad y el modelo mejora significativamente.

❏ Regla de oro: **siempre escala antes de aplicar k-NN**, usando los parámetros calculados sobre el conjunto de entrenamiento.

Elección del hiperparámetro k

El valor de k controla el equilibrio entre sesgo y varianza del modelo. No existe un valor universalmente óptimo: debe ajustarse mediante validación cruzada para cada problema específico.

1

k muy pequeño (ej. $k=1$)

Alta varianza, bajo sesgo. El modelo se adapta perfectamente al entrenamiento pero es muy sensible al ruido y a los *outliers*. Riesgo de sobreajuste.

2

k moderado

Equilibrio óptimo. Generaliza bien a datos nuevos. Se suele elegir mediante validación cruzada k -fold, probando valores impares para evitar empates.

3

k muy grande (ej. $k=N$)

Bajo varianza, alto sesgo. El modelo ignora la estructura local y tiende a predecir siempre la clase mayoritaria. Riesgo de infraajuste.



Naive Bayes: probabilidad al servicio de la clasificación

Naive Bayes es un clasificador probabilístico basado en el **Teorema de Bayes**. Calcula la probabilidad de que un ejemplo pertenezca a cada clase y elige la más probable:

$$P(C \mid x) = \frac{P(x \mid C) \cdot P(C)}{P(x)}$$

Donde **P(C)** es la probabilidad a priori de la clase, **P(x|C)** es la verosimilitud de observar los datos dada la clase, y **P(C|x)** es la probabilidad a posteriori que buscamos maximizar.

La hipótesis «naive»

El adjetivo *naive* (ingenuo) hace referencia a la **hipótesis de independencia condicional**: se asume que, dada la clase, todas las características son independientes entre sí.

Aunque esta hipótesis raramente se cumple en la práctica, el modelo funciona sorprendentemente bien, especialmente en clasificación de texto, análisis de sentimientos y filtrado de spam.

- ❏ La independencia no tiene que ser real para que Naive Bayes funcione bien en la práctica.

Variantes de Naive Bayes

Dependiendo del tipo de datos y del problema, existen distintas variantes del clasificador Naive Bayes que modelan la verosimilitud de forma diferente:



Multinomial NB

Ideal para datos de **conteo de palabras** (bolsa de palabras). Cada característica es la frecuencia de aparición de un término. Es el estándar en clasificación de texto y recuperación de información.



Bernoulli NB

Trabaja con características **binarias** (presencia/ausencia de palabras). Útil cuando el vocabulario es extenso y solo importa si una palabra aparece o no, independientemente de cuántas veces lo haga.



Gaussiano NB

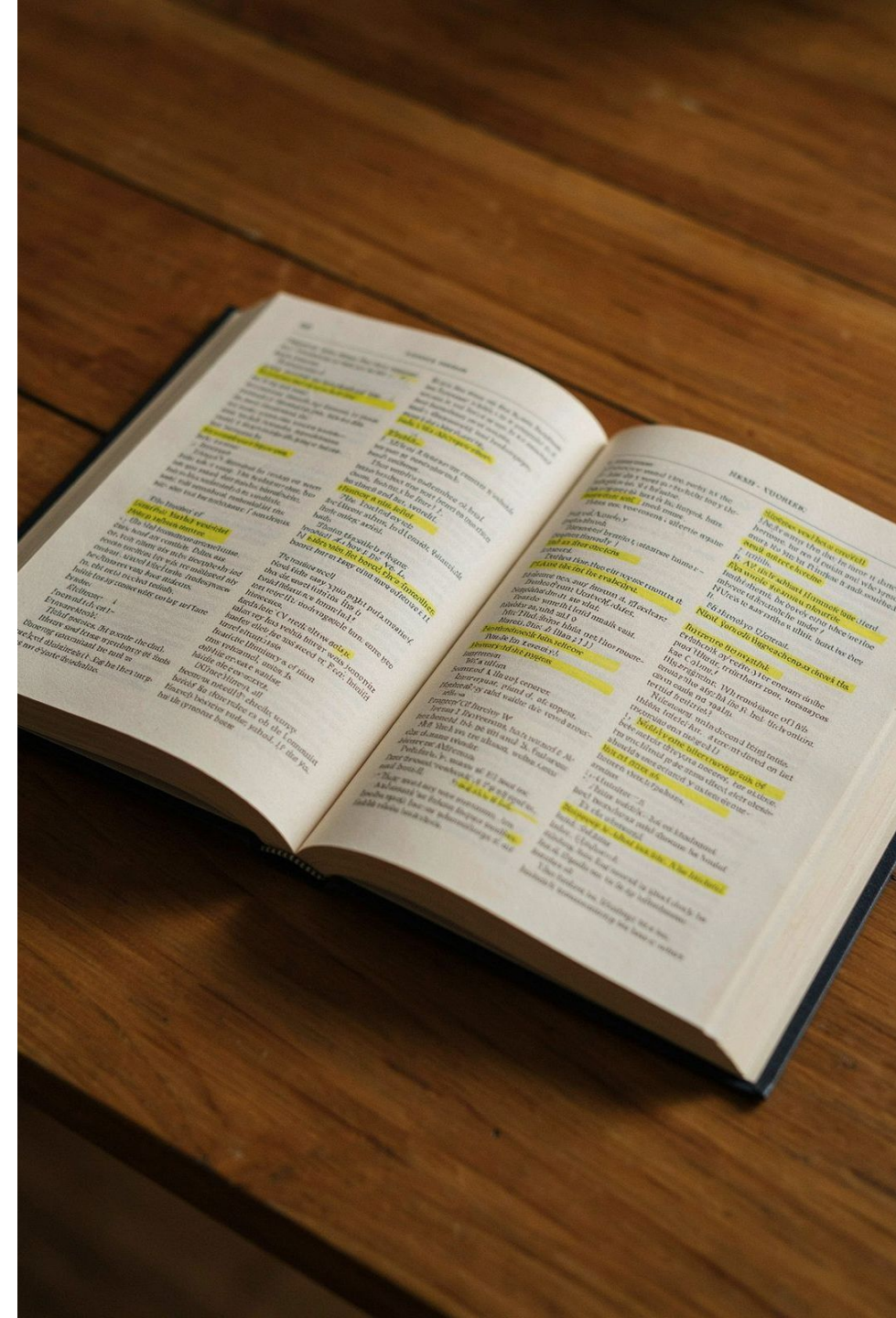
Asume que las características siguen una **distribución normal**. Se usa con variables continuas como medidas físicas o sensores. No es apropiado para datos de texto de forma directa.

Aplicación de Naive Bayes al texto

El clasificador Naive Bayes Multinomial es el enfoque más habitual para clasificar documentos de texto. El proceso completo sigue estos pasos:



Un aspecto crucial es aplicar **suavizado de Laplace** ($\alpha=1$) para evitar que la aparición de una palabra desconocida en test anule completamente la probabilidad de una clase.



Naive Bayes en la práctica: filtrado de spam

El caso de uso más famoso

El filtrado de correo spam fue uno de los primeros éxitos prácticos de Naive Bayes. El clasificador aprende a distinguir spam de correo legítimo a partir de la frecuencia de palabras características de cada categoría.

- **Spam:** «oferta», «gratis», «urgente», «haga clic»
- **Ham:** «reunión», «informe», «adjunto», «proyecto»

Con millones de ejemplos de entrenamiento, los filtros modernos alcanzan tasas de precisión superiores al 99%.

Análisis de sentimientos

Otra aplicación destacada es la clasificación de reseñas como positivas o negativas. El modelo estima la probabilidad de cada polaridad dadas las palabras del texto.

Clasificación temática

Asignar categorías (deportes, política, tecnología) a noticias o artículos de forma automática. Naive Bayes ofrece un **baseline sólido y extremadamente rápido** para estos problemas.

- ❗ En texto, Naive Bayes suele superar a k-NN porque evita el costoso cálculo de distancias en espacios de muy alta dimensión.

La maldición de la dimensionalidad en k-NN

Cuando se aplica k-NN a texto, los documentos se representan como vectores de miles de dimensiones (una por palabra del vocabulario). Esto provoca un fenómeno conocido como la **maldición de la dimensionalidad**:

1

Las distancias se igualan

En alta dimensión, todos los puntos tienden a estar a distancias similares entre sí, perdiendo significado la noción de «vecino cercano».

2

Coste computacional alto

Calcular distancias entre el ejemplo de test y todos los ejemplos de entrenamiento es muy costoso cuando el espacio tiene miles de dimensiones.

3

Sparsity del texto

Los vectores de texto son muy dispersos (la mayoría de palabras no aparecen en cada documento), lo que distorsiona aún más las métricas de distancia estándar.

4

Soluciones parciales

Usar distancia coseno, reducir dimensionalidad con PCA o LSA, o emplear estructuras como KD-trees para acelerar la búsqueda de vecinos.

Comparativa: k-NN vs. Naive Bayes

Ambos algoritmos tienen perfiles de fortalezas y debilidades muy distintos. Elegir entre ellos depende del tipo de datos, el tamaño del conjunto y los recursos disponibles:

Criterio	k-NN	Naive Bayes
Tipo de aprendizaje	Lazy (sin entrenamiento)	Eager (construye un modelo)
Coste en predicción	Alto (busca en todos los datos)	Muy bajo (multiplicación de prob.)
Alta dimensionalidad	Problemática (maldición dim.)	Maneja bien con texto disperso
Interpretabilidad	Baja (caja negra local)	Alta (probabilidades explicables)
Hipótesis sobre datos	Ninguna (no paramétrico)	Independencia condicional
Mejor uso en texto	Datasets pequeños, baja dim.	Clasificación de documentos



Conclusiones y criterios de elección

k-NN y Naive Bayes son dos pilares fundamentales del aprendizaje automático clásico. Comprender sus mecanismos internos es esencial para tomar decisiones informadas en proyectos reales.

Elige k-NN cuando...

- El conjunto de datos es **pequeño** y cabe en memoria
- Las fronteras de decisión son **no lineales y complejas**
- Las características son **numéricas y están escaladas**
- Necesitas un modelo sin suposiciones sobre la distribución

Elige Naive Bayes cuando...

- Trabajas con **texto o datos categóricos** dispersos
- El dataset es **grande** y la velocidad es prioritaria
- Necesitas **probabilidades calibradas** como salida
- Quieres un **baseline sólido** rápido de implementar



Ambos modelos son excelentes puntos de partida antes de recurrir a algoritmos más complejos como SVM o redes neuronales.